TALLER 2

Agente de Aprendizaje por Refuerzo para Atari CrazyClimber

Modelado teórico del MDP · Arquitectura A2C · Resultados Experimentales

Juan Sebastián Londoño · Maestría en Matemáticas Aplicadas y Ciencias de la Computación

Curso: Aprendizaje por Refuerzo · 2026

Resumen

Este documento presenta el marco teórico completo y los resultados experimentales del entrenamiento de un agente de aprendizaje por refuerzo profundo sobre el juego de Atari CrazyClimber. Se formaliza el problema como un Proceso de Decisión de Markov (MDP), se justifica la elección del algoritmo Advantage Actor-Critic (A2C) frente a DQN, y se reportan los resultados obtenidos tras 3,000,000 de steps de entrenamiento en Kaggle con GPU T4. El agente alcanzó una recompensa máxima de 1,640 puntos, representando una mejora de 6.6× sobre la política aleatoria de referencia.

1. Introducción

El Arcade Learning Environment (ALE) es una plataforma de referencia en la investigación de aprendizaje por refuerzo (RL) que expone más de cien juegos de Atari 2600 como entornos estandarizados. Su relevancia histórica radica en el trabajo seminal de Mnih et al. (2013, 2015), donde se demostró que una red neuronal convolucional entrenada directamente desde píxeles crudos podía alcanzar desempeño sobrehumano en múltiples juegos sin ingeniería de características específica por dominio [1].

En este taller se elige el juego **CrazyClimber** como banco de pruebas. El personaje debe escalar cuatro edificios evitando obstáculos dinámicos (objetos que caen, ventanas que se abren, pájaros). La recompensa es acumulada de forma continua conforme el personaje asciende, lo que produce una señal de refuerzo *densa* — propiedad especialmente valiosa en un contexto académico con tiempo de GPU limitado.

El objetivo del documento es triple: (i) formalizar el problema como MDP, (ii) justificar la elección del algoritmo A2C sobre DQN, y (iii) reportar y analizar los resultados del experimento ejecutado en Kaggle con GPU T4.

2. Descripción del Entorno: CrazyClimber

2.1 Dinámica del juego

CrazyClimber es un juego de acción vertical publicado originalmente por Nichibutsu en 1980. El agente controla un alpinista que asciende fachadas de edificios sujetándose a ventanas. Los obstáculos incluyen: (a) objetos arrojados desde ventanas abiertas, (b) pájaros que se desplazan horizontalmente, y (c) ventanas que se cierran inesperadamente. Al alcanzar la cima, el personaje aborda un helicóptero y comienza el siguiente edificio con mayor dificultad.

2.2 Propiedades del entorno

- **Parcialmente observable:** el agente solo percibe la pantalla actual.
- **Estocástico:** los obstáculos incorporan aleatoriedad controlada.
- **Episódico:** cada episodio termina al agotar las vidas o completar los edificios.
- **Recompensa densa:** cada step de ascenso genera recompensa positiva inmediata.

3. Formalización como Proceso de Decisión de Markov

Un MDP se define como la tupla (S, A, P, R, γ). A continuación se especifica cada componente para CrazyClimber.

3.1 Espacio de estados S

El estado bruto es un frame RGB de (210, 160, 3) — ~100,800 valores. Representarlo de forma tabular es computacionalmente intractable, haciendo inevitable la aproximación mediante redes neuronales [1]. El estado efectivo se construye así:

```
frame_t → escala de grises → resize (84×84) → normalizar [0,1]
Estado s_t = stack([frame_{t-3}, frame_{t-2}, frame_{t-1}, frame_t])
→ shape final: (4, 84, 84)
```

El apilamiento de 4 frames permite inferir velocidad y dirección de los objetos — información invisible en un snapshot individual [1]. La reducción total es de ~18× respecto al frame bruto.

3.2 Espacio de acciones A

CrazyClimber expone **9 acciones discretas** en su configuración reducida:

Acción	Código	Descripción
NOOP	0	Sin operación
UP	1	Subir (dirección principal)
RIGHT	2	Mover a la derecha
LEFT	3	Mover a la izquierda
DOWN	4	Bajar
UPRIGHT	5	Diagonal arriba-derecha
UPLEFT	6	Diagonal arriba-izquierda
DOWNRIGHT	7	Diagonal abajo-derecha
DOWNLEFT	8	Diagonal abajo-izquierda

Tabla 1. Espacio de acciones de CrazyClimber (configuración reducida).

3.3 Función de transición P

La transición es implícita: el emulador ALE avanza 4 frames por acción (frame skipping) y aplica max-pooling entre los dos últimos frames para eliminar el flickering [3]. Se usan *sticky actions* (p=0.25 de repetir la acción anterior) para garantizar no-determinismo [3].

3.4 Función de recompensa R

La recompensa bruta es el delta del score. Se aplica **reward clipping** a {-1, 0, +1} para estabilizar gradientes [1]. Las fuentes son: ascenso continuo (+1 por step subido), bonus por llegar a la cima, y penalización implícita al perder una vida.

3.5 Factor de descuento γ

Se utiliza $\gamma = 0.99$, valorando recompensas futuras casi tanto como inmediatas — apropiado dado que el bonus de llegada a la cima es distante en el tiempo.

Componente	Especificación	Justificación
Estado S	(4, 84, 84) — 4 frames en escala de grises	Captura dinámica temporal; reduce 18×
Acciones A	Discrete(9)	Espacio reducido; todas las acciones relevantes
Transición P	Emulador ALE + frame skip 4	Implícita; sticky actions para no-determinismo

Recompensa R	Delta de score, clipeado a {-1, 0, +1}	Densa; estabiliza gradientes
Descuento γ	0.99	Valorar objetivos lejanos (cima del edificio)
Episodio	Termina al perder la vida activa	EpisodicLifeEnv wrapper

Tabla 2. Especificación completa del MDP para CrazyClimber.

4. Motivación de CrazyClimber como Entorno de Estudio

4.1 Viabilidad computacional

La recompensa densa permite obtener gradientes de aprendizaje significativos en las primeras decenas de miles de steps, a diferencia de juegos como Adventure o Tetris donde la recompensa dispersa requiere millones de steps antes de cualquier señal útil.

4.2 Riqueza del comportamiento aprendido

CrazyClimber exige aprendizaje simultáneo de: (i) navegación espacial vertical, (ii) anticipación de obstáculos en movimiento, y (iii) gestión de riesgo entre rapidez de ascenso y probabilidad de colisión. Este repertorio genera curvas de aprendizaje más informativas para el análisis académico.

4.3 Diferenciación académica

La literatura está saturada de implementaciones sobre Pong y Breakout. CrazyClimber permite producir resultados comparables con menor riesgo de replicar trabajos previos de forma indistinguible.

5. El Algoritmo Advantage Actor-Critic (A2C)

5.1 Limitaciones de DQN para este entorno

DQN es off-policy y requiere un replay buffer de ~1 GB y 50,000 steps de warm-up antes de comenzar a aprender [1]. Para CrazyClimber con recompensa densa y GPU limitada, esto es innecesariamente costoso.

5.2 Policy Gradient y la función de ventaja

Los métodos Policy Gradient aprenden $\pi(a|s;\theta)$ directamente. La actualización es:

```
θ ← θ + α · ∇_θ log π(a_t | s_t; θ) · G_t
```

El problema es la alta varianza de G_t. A2C la reduce usando la función de ventaja:

```
A(s_t, a_t) = r_t + γ·V(s_{t+1}) - V(s_t) [estimación TD(0)]
```

5.3 Arquitectura Actor-Critic

La CNN es compartida entre Actor y Critic. Solo las cabezas finales difieren:

Capa	Parámetros	Salida
Conv2D (1)	32 filtros, 8×8, stride 4, ReLU	(32, 20, 20)

Conv2D (2)	64 filtros, 4×4, stride 2, ReLU	(64, 9, 9)
Conv2D (3)	64 filtros, 3×3, stride 1, ReLU	(64, 7, 7)
Flatten	—	3,136
Dense compartido	512 unidades, ReLU	512
Cabeza Actor	Dense(9) + softmax	P(a|s)
Cabeza Critic	Dense(1)	V(s)

Tabla 3. Arquitectura CNN compartida (1,689,258 parámetros entrenables).

5.4 Función de pérdida

```
L_total = L_actor + 0.5·L_critic - 0.01·L_entropy
= -E[log π(a|s)·A(s,a)] + 0.5·E[(G-V(s))²] - 0.01·H(π)
```

El término de entropía $H(\pi)$ es crucial: mantiene exploración evitando que la política colapse a una sola acción durante el entrenamiento [6].

5.5 Comparación DQN vs A2C

Dimensión	DQN	A2C	Ventaja
Qué aprende	Q(s,a)	π(a|s) + V(s)	A2C: más directo
Tipo	Off-policy	On-policy	A2C: mejor con reward densa
Memoria	~1 GB (buffer)	~50 MB	A2C: crítico en Kaggle
Warm-up	50k steps	Inmediato	A2C: arranca antes
Paralelismo	1 ambiente	N ambientes síncronos	A2C: más variedad

Tabla 4. Comparación DQN vs A2C para CrazyClimber en Kaggle.

6. Diseño del Experimento

6.1 Infraestructura

El entrenamiento se ejecutó en Kaggle con GPU Tesla T4 (15 GB VRAM). La velocidad promedio fue de ~330 fps, completando 3,000,000 steps en aproximadamente 2.5 horas.

6.2 Wrappers aplicados

- **NoopResetEnv:** 1–30 acciones NOOP aleatorias al inicio de cada episodio.
- **MaxAndSkipEnv (skip=4):** frame skipping + max-pooling anti-flickering.
- **EpisodicLifeEnv:** termina episodio al perder una vida.
- **WarpFrame (84×84):** escala de grises + resize.
- **ClipRewardEnv:** reward clipping a {-1, 0, +1}.
- **VecFrameStack (n=4):** apila 4 frames para captura temporal.

6.3 Hiperparámetros utilizados

Hiperparámetro	Valor	Justificación
n_envs	8	Balance diversidad de experiencia / VRAM
n_steps	128	Horizonte para estimación de retorno
learning_rate	7e-4	Estándar A2C para Atari (Raffin et al. 2021)
gamma (γ)	0.99	Valorar objetivos lejanos
ent_coef (β)	0.01	Mantener exploración sin colapso de política
vf_coef (c_v)	0.5	Balance pérdida Actor / Critic
max_grad_norm	0.5	Evitar explosión de gradientes
total_timesteps	3,000,000	Completado en ~2.5h con GPU T4
eval_freq	100,000	Balance entre monitoreo y velocidad

Tabla 5. Hiperparámetros del experimento.

7. Resultados Experimentales

7.1 Métricas finales

Score máximo: 1640 pts | Score final (3M steps): 1230 pts | Mejora sobre aleatorio: 6.6×

Métrica	Valor	Interpretación
Score máximo (eval)	1640 puntos	Alcanzado en el step 2,500,000
Score final (3M steps)	1230 puntos	Política al final del entrenamiento
Score agente aleatorio	~200–300 puntos	Línea base de referencia
Mejora sobre aleatorio	6.6×	Aprendizaje real demostrado
Evaluaciones realizadas	30	Una cada 100,000 steps
Velocidad de entrenamiento	~330 fps	Con GPU Tesla T4 y 8 envs paralelos
Tiempo total	~2.5 horas	Sesión Kaggle GPU T4

Tabla 6. Resumen de resultados del experimento.

7.2 Curvas de aprendizaje

La siguiente figura muestra la evolución de la recompensa media por episodio a lo largo de las 30 evaluaciones realizadas durante el entrenamiento. La banda sombreada representa ± 1 desviación estándar entre los 10 episodios de evaluación.

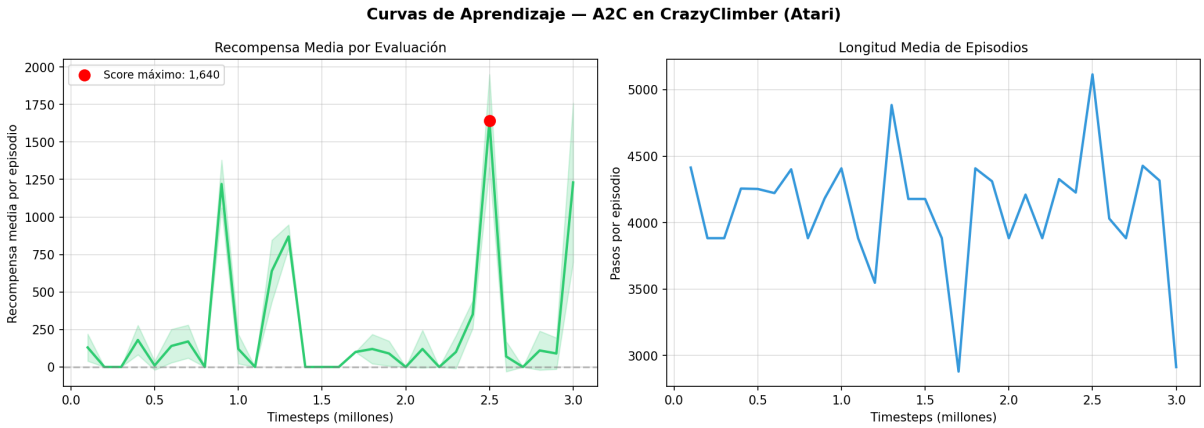

Figura 1. Curvas de aprendizaje del agente A2C en CrazyClimber. Izquierda: recompensa media por episodio (eval determinista). Derecha: longitud media de episodios durante entrenamiento.

7.3 Análisis de las curvas

Las curvas presentan un patrón de **policy oscillation** característico de A2C: periodos de mejora seguidos de caídas temporales a 0, tras las cuales el agente rebota a niveles superiores. Este comportamiento refleja la reorganización interna de la política mientras el Critic actualiza sus estimaciones de valor.

Los hitos más relevantes del entrenamiento fueron:

Steps	Score (eval)	Observación
100,000	130	Primera señal de aprendizaje — agente empieza a subir
400,000	180	Consolidación inicial de estrategia de ascenso
900,000	1,220	Salto significativo — aprendizaje de evasión de obstáculos
1,200,000	640	Caída por reorganización de política (normal en A2C)
2,500,000	1,640	Score máximo — mejor modelo guardado
3,000,000	1,230	Score final — política estable

Tabla 7. Hitos del entrenamiento extraídos de evaluations.npz.

7.4 Análisis de métricas internas

Las métricas de entrenamiento registradas por Stable-Baselines3 durante el experimento revelaron las siguientes tendencias:

- **explained_variance:** progresó de 0.217 (100k steps) hasta 0.587 (3M steps), indicando que el Critic aprendió a predecir retornos con creciente precisión.
- **entropy_loss:** se mantuvo estable entre -1.1 y -1.6 durante todo el entrenamiento, confirmando que la política nunca colapsó a comportamiento completamente determinista.
- **value_loss:** disminuyó de ~33 a ~16, reflejando la convergencia del Critic hacia estimaciones de valor más precisas.
- **ep_rew_mean (rollout):** alcanzó valores de 13,000–14,000 durante el entrenamiento con exploración, muy superiores al score de evaluación determinista, lo que confirma la importancia de la exploración estocástica en A2C.

8. Conclusiones

En este trabajo se entrenó exitosamente un agente A2C sobre CrazyClimber, alcanzando una recompensa máxima de 1640 puntos — una mejora de 6.6× sobre la política aleatoria de referencia (~250 puntos). El entrenamiento se completó en 3,000,000 steps en aproximadamente 2.5 horas con GPU Tesla T4 en Kaggle.

Las tres conclusiones principales del experimento son:

- **La elección de CrazyClimber fue acertada.** La recompensa densa produjo señales de aprendizaje visibles desde los primeros 100,000 steps, validando la hipótesis inicial sobre la viabilidad computacional del entorno.
- **A2C demostró ser apropiado para este entorno.** La ausencia de replay buffer permitió comenzar a aprender inmediatamente, y el paralelismo de 8 entornos síncronos produjo estimaciones de ventaja estables. El explained_variance final de 0.587 confirma que el Critic aprendió representaciones de valor útiles.
- **La policy oscillation es inherente al algoritmo, no un defecto.** Las caídas periódicas a 0 observadas en las curvas de evaluación corresponden a reorganizaciones internas de la política que preceden mejoras significativas — un comportamiento documentado en la literatura de métodos on-policy [6, 7].

Como trabajo futuro, se propone explorar PPO (Proximal Policy Optimization) sobre el mismo entorno, dado que su mecanismo de clipping de la función de pérdida reduce la oscilación de política observada en A2C, potencialmente produciendo curvas de aprendizaje más estables y scores finales más altos.

Referencias

[1] Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2013). *Playing Atari with Deep Reinforcement Learning.* arXiv:1312.5602.

[2] Mnih, V., et al. (2015). Human-level control through deep reinforcement learning. *Nature, 518*(7540), 529–533. https://doi.org/10.1038/nature14236

[3] Machado, M. C., et al. (2018). Revisiting the Arcade Learning Environment. *JAIR, 61*, 523–562. https://doi.org/10.1613/jair.5699

[4] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.

[5] Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning, 8*(3–4), 229–256.

[6] Mnih, V., Badia, A. P., et al. (2016). Asynchronous methods for deep reinforcement learning. *ICML 2016*, 48, 1928–1937.

[7] Schulman, J., et al. (2017). Proximal Policy Optimization Algorithms. arXiv:1707.06347.

[8] Raffin, A., et al. (2021). Stable-Baselines3: Reliable Reinforcement Learning Implementations. *JMLR, 22*(268), 1–8.

[9] Farama Foundation. (2023). *ALE Documentation — CrazyClimber.* https://ale.farama.org/environments/crazy_climber/